In [1]:
```
!python -m pip install numpy pandas matplotlib scikit-learn gensim
```

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: numpy in c:\users\icon\appdata\roaming\python\pyth
on311\site-packages (1.24.4)
Requirement already satisfied: pandas in c:\users\icon\appdata\roaming\python\pyt
hon311\site-packages (1.5.3)
Requirement already satisfied: matplotlib in c:\programdata\anaconda3\lib\site-pa
ckages (3.8.0)
Requirement already satisfied: scikit-learn in c:\programdata\anaconda3\lib\site-
packages (1.2.2)
Requirement already satisfied: gensim in c:\programdata\anaconda3\lib\site-packag
es (4.3.0)
Requirement already satisfied: python-dateutil>=2.8.1 in c:\programdata\anaconda3
\lib\site-packages (from pandas) (2.8.2)
Requirement already satisfied: pytz>=2020.1 in c:\programdata\anaconda3\lib\site-
packages (from pandas) (2023.3.post1)
Requirement already satisfied: contourpy>=1.0.1 in c:\programdata\anaconda3\lib\s
ite-packages (from matplotlib) (1.2.0)
Requirement already satisfied: cycler>=0.10 in c:\programdata\anaconda3\lib\site-
packages (from matplotlib) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in c:\programdata\anaconda3\lib
\site-packages (from matplotlib) (4.25.0)
Requirement already satisfied: kiwisolver>=1.0.1 in c:\programdata\anaconda3\lib
\site-packages (from matplotlib) (1.4.4)
Requirement already satisfied: packaging>=20.0 in c:\programdata\anaconda3\lib\si
te-packages (from matplotlib) (23.1)
Requirement already satisfied: pillow>=6.2.0 in c:\programdata\anaconda3\lib\site
-packages (from matplotlib) (10.2.0)
Requirement already satisfied: pyparsing>=2.3.1 in c:\programdata\anaconda3\lib\s
ite-packages (from matplotlib) (3.0.9)
Requirement already satisfied: scipy>=1.3.2 in c:\users\icon\appdata\roaming\pyth
on\python311\site-packages (from scikit-learn) (1.10.1)
Requirement already satisfied: joblib>=1.1.1 in c:\programdata\anaconda3\lib\site
-packages (from scikit-learn) (1.2.0)
Requirement already satisfied: threadpoolctl>=2.0.0 in c:\programdata\anaconda3\l
ib\site-packages (from scikit-learn) (2.2.0)
Requirement already satisfied: smart-open>=1.8.1 in c:\programdata\anaconda3\lib
\site-packages (from gensim) (5.2.1)
Requirement already satisfied: FuzzyTM>=0.4.0 in c:\users\icon\appdata\roaming\py
thon\python311\site-packages (from gensim) (2.0.9)
Requirement already satisfied: pyfume in c:\users\icon\appdata\roaming\python\pyt
hon311\site-packages (from FuzzyTM>=0.4.0->gensim) (0.3.4)
Requirement already satisfied: six>=1.5 in c:\programdata\anaconda3\lib\site-pack
ages (from python-dateutil>=2.8.1->pandas) (1.16.0)
Requirement already satisfied: simpful==2.12.0 in c:\users\icon\appdata\roaming\p
ython\python311\site-packages (from pyfume->FuzzyTM>=0.4.0->gensim) (2.12.0)
Requirement already satisfied: fst-pso==1.8.1 in c:\users\icon\appdata\roaming\py
thon\python311\site-packages (from pyfume->FuzzyTM>=0.4.0->gensim) (1.8.1)
Requirement already satisfied: miniful in c:\users\icon\appdata\roaming\python\py
thon311\site-packages (from fst-pso==1.8.1->pyfume->FuzzyTM>=0.4.0->gensim) (0.0.
6)
```

In [2]:
```
import numpy
import pandas
import matplotlib
import sklearn
import gensim
```

```
print("All libraries installed successfully!")
```

```
All libraries installed successfully!
```

In [3]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import re

from sklearn.datasets import fetch_20newsgroups
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.metrics.pairwise import cosine_similarity

from gensim.models import Word2Vec

print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

In [4]:
```
categories = [
    'rec.sport.baseball',
    'sci.space',
    'comp.graphics',
    'talk.politics.misc'
]

data = fetch_20newsgroups(
    subset='all',
    categories=categories,
    remove=('headers', 'footers', 'quotes')
)

X = data.data
y = data.target

print("Total Documents:", len(X))
print("Categories:", data.target_names)
```

```
Total Documents: 3729
Categories: ['comp.graphics', 'rec.sport.baseball', 'sci.space', 'talk.politics.m
isc']
```

In [5]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Training Documents:", len(X_train))
print("Testing Documents:", len(X_test))
```

```
Training Documents: 2983
Testing Documents: 746
```

In [6]:
```
bow = CountVectorizer(max_features=3000)

X_train_bow = bow.fit_transform(X_train)
```

```
X_test_bow = bow.transform(X_test)

print("Bag-of-Words Representation:")
print(X_train_bow.shape)
print(X_train_bow.toarray()[:3, :10])
```

```
Bag-of-Words Representation:
(2983, 3000)
[[0 0 0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0 0 0]]
```

In [7]:
```
tfidf = TfidfVectorizer(max_features=3000)

X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)

print("\nTF-IDF Representation:")
print(X_train_tfidf.shape)
print(X_train_tfidf.toarray()[:3, :10])
```

```
TF-IDF Representation:
(2983, 3000)
[[0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]]
```

In [8]:
```
def tokenize(text):
    return re.findall(r'\b[a-z]+\b', text.lower())


tokenized_sentences = [
    tokenize(text) for text in X_train
]

w2v_model = Word2Vec(
    sentences=tokenized_sentences,
    vector_size=100,
    window=5,
    min_count=2,
    workers=4,
    epochs=10
)

print("Word2Vec Model Created Successfully!")
print("Vocabulary Size:", len(w2v_model.wv))
```

```
Word2Vec Model Created Successfully!
Vocabulary Size: 16364
```

In [9]:
```
def get_document_vector(text):

    words = tokenize(text)

    vectors = [
        w2v_model.wv[word]
        for word in words
        if word in w2v_model.wv
    ]

    if len(vectors) == 0:
```

```
        return np.zeros(100)

    return np.mean(vectors, axis=0)


X_train_w2v = np.array([
    get_document_vector(text) for text in X_train
])

X_test_w2v = np.array([
    get_document_vector(text) for text in X_test
])

print("Word2Vec Representation:")
print(X_train_w2v.shape)
```

```
Word2Vec Representation:
(2983, 100)
```

In [10]:

```
text1 = "NASA launched a rocket into space"
text2 = "The spacecraft was sent into orbit"
text3 = "The baseball team won the match"

texts = [text1, text2, text3]

# Bag-of-Words similarity
bow_vectors = bow.transform(texts)
bow_similarity = cosine_similarity(bow_vectors)[0, 1]

# TF-IDF similarity
tfidf_vectors = tfidf.transform(texts)
tfidf_similarity = cosine_similarity(tfidf_vectors)[0, 1]

# Word2Vec similarity
w2v_vectors = np.array([
    get_document_vector(text) for text in texts
])

w2v_similarity = cosine_similarity(w2v_vectors)[0, 1]

print("\nTEXT SIMILARITY COMPARISON")

print("Bag-of-Words:", round(bow_similarity, 4))
print("TF-IDF:", round(tfidf_similarity, 4))
print("Word2Vec:", round(w2v_similarity, 4))
```

```
TEXT SIMILARITY COMPARISON
Bag-of-Words: 0.1826
TF-IDF: 0.1069
Word2Vec: 0.674
```

In [11]:

```
models = {
    "Bag-of-Words": (X_train_bow, X_test_bow),
    "TF-IDF": (X_train_tfidf, X_test_tfidf),
    "Word2Vec": (X_train_w2v, X_test_w2v)
}

results = {}

print("\nCLASSIFICATION RESULTS")
```

```
for name, (train_data, test_data) in models.items():

    classifier = LogisticRegression(max_iter=1000)

    classifier.fit(train_data, y_train)

    predictions = classifier.predict(test_data)

    accuracy = accuracy_score(y_test, predictions)

    results[name] = accuracy

    print(name, "Accuracy:",
          round(accuracy * 100, 2), "%")
```

```
CLASSIFICATION RESULTS
Bag-of-Words Accuracy: 83.51 %
TF-IDF Accuracy: 86.19 %
Word2Vec Accuracy: 78.42 %
```

In [12]:
```
comparison = pd.DataFrame({
    "Technique": results.keys(),
    "Accuracy (%)": [
        round(value * 100, 2)
        for value in results.values()
    ]
})

print("\nFINAL COMPARISON TABLE")
print(comparison.to_string(index=False))
```

```
FINAL COMPARISON TABLE
   Technique  Accuracy (%)
Bag-of-Words         83.51
      TF-IDF         86.19
    Word2Vec         78.42
```

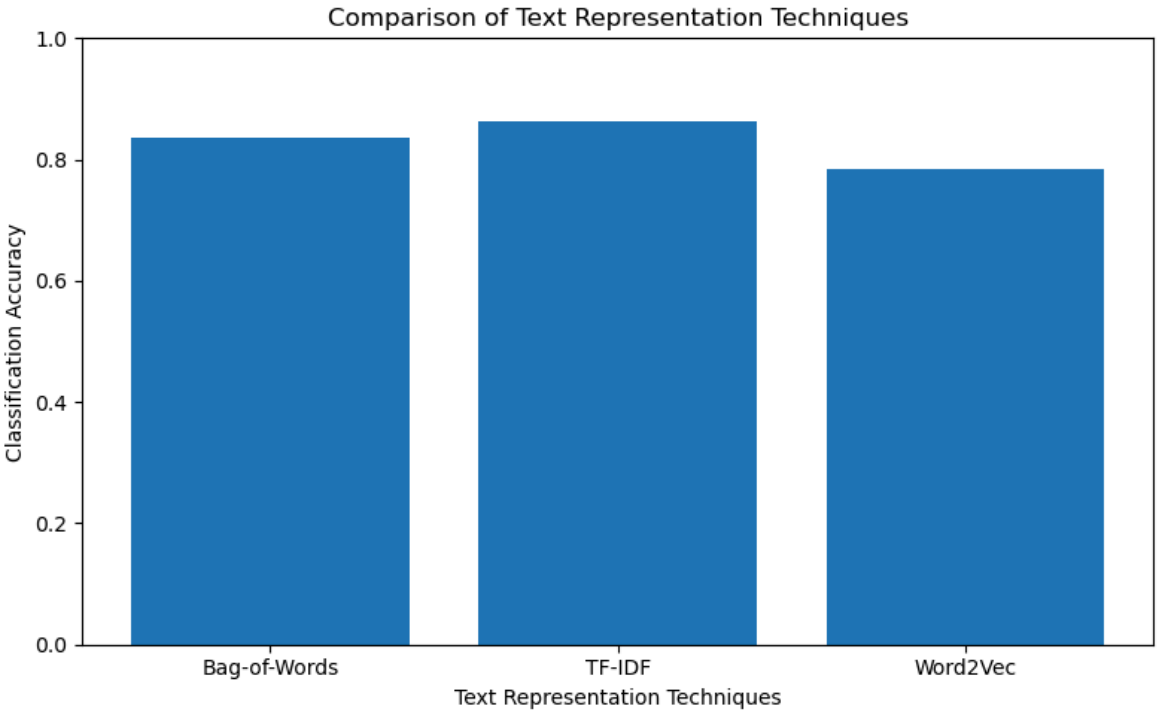
In [13]:
```
plt.figure(figsize=(8, 5))

plt.bar(results.keys(), results.values())

plt.title("Comparison of Text Representation Techniques")
plt.xlabel("Text Representation Techniques")
plt.ylabel("Classification Accuracy")

plt.ylim(0, 1)

plt.tight_layout()
plt.show()
```


Comparison of Text Representation Techniques
Classification Accuracy
1.0
0.8
0.6
0.4
0.2
0.0
Bag-of-Words
TF-IDF
Word2Vec
Text Representation Techniques